

RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering

Academic Year: 2025 – 2026

MINI PROJECT REPORT

On

Construction Site Safety Monitoring using Computer Vision and Image Processing

Individual Report — Algorithm A: HOG + SVM Multi-Class PPE Detector

Submitted by

SUDHARSHAN KRISHNAA L K

Roll No: 230701350

Section: [CSE-X]

Email: 230701350@rajalakshmi.edu.in

Date of Submission: [DD Month YYYY]

TABLE OF CONTENTS

1. Introduction	3
1.1 Background and Motivation	
1.2 Problem Statement	
1.3 Objectives	
1.4 Scope of the Report	
2. Literature Review	4
2.1 Traditional Feature-Based Methods	
2.2 Deep Learning Approaches	
2.3 Existing Method Studies	
2.4 Gap Identified	
3. Dataset Description	5
3.1 Hard Hat Workers Dataset (Kaggle)	
3.2 Annotation Format and Conversion	
3.3 Data Split	
4. Preprocessing Methodology	6
4.1 Resizing	
4.2 Gaussian Denoising	
4.3 CLAHE Contrast Enhancement	
4.4 Before vs After Visualisation	
5. Algorithm: HOG + SVM	8
5.1 Algorithm Overview	
5.2 Stage 1 — HOG Feature Extraction	
5.3 Stage 2 — SVM Classification	
5.4 Handling Class Imbalance (SMOTE)	
5.5 Implementation Details	
5.6 Algorithm Flowchart	
6. Results and Analysis	11
6.1 Quantitative Results	
6.2 Effect of Preprocessing	
6.3 Confusion Matrix Analysis	
6.4 ROC Curve Analysis	
7. Discussion	13

8. Conclusion and Future Work	14
References	15
Appendix A — Source Code	16
Appendix B — Sample Output Screenshots	21

LIST OF TABLES AND FIGURES

Tables

- Table 1: Dataset Statistics
- Table 2: Train / Test Split
- Table 3: HOG Parameter Configuration
- Table 4: SVM Implementation Configuration
- Table 5: HOG + SVM Test Set Results
- Table 6: Preprocessing Ablation Study

Figures

- Figure 1: Dataset class distribution (bar chart)
- Figure 2: COCO annotation to classification conversion pipeline
- Figure 3: Before vs After preprocessing — 3-stage comparison
- Figure 4: HOG gradient visualisation (grayscale vs HOG image)
- Figure 5: Algorithm architecture flowchart
- Figure 6: Confusion matrix heatmap
- Figure 7: ROC curve — no_helmet detection
- Figure 8: Streamlit web application screenshots

1. INTRODUCTION

1.1 Background and Motivation

Construction sites are among the most hazardous work environments in the world. According to the International Labour Organization (ILO), the construction industry accounts for approximately 17% of all occupational fatalities globally despite employing only 7% of the workforce. Head injuries are a leading cause of construction-related fatalities, and the consistent wearing of safety helmets has been shown to reduce the risk of fatal head injury by over 85%. Despite mandatory personal protective equipment (PPE) regulations enforced in most countries, non-compliance remains a persistent challenge due to the difficulty of manually monitoring large construction sites with limited supervisory staff.

Traditional compliance monitoring relies on human supervisors physically patrolling the site or reviewing CCTV footage after incidents have occurred — both approaches are reactive, labour-intensive, and error-prone. The emergence of affordable high-resolution CCTV infrastructure and advances in computer vision and machine learning have created an opportunity for proactive, automated real-time safety monitoring that operates continuously without fatigue or distraction.

This project addresses the problem of automated helmet detection in construction site imagery using a classical computer vision approach: Histogram of Oriented Gradients (HOG) for feature extraction combined with a Support Vector Machine (SVM) for binary classification. This approach was deliberately chosen over deep learning methods to investigate the feasibility of a lightweight, interpretable, and computationally inexpensive system that can be deployed on standard hardware without GPU acceleration — a realistic constraint for small and medium construction firms.

1.2 Problem Statement

Construction sites are inherently high-risk environments where worker safety depends on strict adherence to Personal Protective Equipment (PPE) regulations, especially safety helmets. Despite established guidelines, ensuring consistent compliance remains a major challenge. Manual monitoring methods such as on-site supervision and CCTV review are inefficient and impractical for large-scale environments. Head injuries account for a major proportion of construction-related fatalities. Although helmets can reduce fatal injury risk by over 85%, real-world compliance remains inconsistent due to lack of continuous monitoring. With advancements in computer vision, there is an opportunity to automate safety monitoring. However, most existing solutions rely on deep learning, which requires high

computational resources and complex deployment, making them less accessible for small and medium-scale firms.

Therefore, there is a need for a lightweight and efficient system that can automatically detect whether a worker is wearing a safety helmet using standard computing resources.

1.3 Objectives

The specific objectives of this project are:

1. To design and implement a complete image preprocessing pipeline (resizing, Gaussian denoising, CLAHE contrast enhancement) tailored to construction site imagery.
2. To extract discriminative HOG feature vectors from preprocessed images and analyse their dimensionality and spatial distribution.
3. To train a Support Vector Machine classifier with RBF kernel on HOG features, applying SMOTE oversampling to handle the severe class imbalance in the available dataset.
4. To evaluate the trained model on a held-out test set using accuracy, precision, recall, F1-score, and ROC curve metrics.
5. To develop a complete prediction system supporting both single image and video input, with an interactive Streamlit web interface for demonstration.
6. To document the limitations arising from dataset imbalance and propose directions for improvement.

1.4 Scope of the Report

This report presents the complete end-to-end pipeline, starting from raw dataset collection and preprocessing to model development and deployment as a functional web application. The system focuses specifically on binary classification, determining whether a worker is wearing a helmet (*helmet*) or not (*no_helmet*) using pre-cropped images of individual persons.

The scope is limited to classification only; advanced tasks such as multi-person detection, object localization, and bounding box regression are not considered in this project. The implementation is carried out using Python 3.10, leveraging libraries such as OpenCV for image processing, scikit-image for feature extraction, scikit-learn for machine learning, and imbalanced-learn for handling class imbalance.

Additionally, the system is designed to operate entirely on classical computer vision and machine learning techniques, without the use of deep learning frameworks like TensorFlow or PyTorch, ensuring a lightweight and CPU-efficient solution suitable for real-world deployment.

2. LITERATURE REVIEW

Helmet detection in construction environments has gained significant attention as an important safety-focused computer vision problem in recent years. This section reviews key prior works related to classical feature-based approaches, particularly the Histogram of Oriented Gradients (HOG) combined with Support Vector Machine (SVM), along with comparisons to modern deep learning-based methods.

2.1 Traditional Feature-Based Methods

Before the deep learning era, object detection and classification systems relied heavily on hand-crafted feature descriptors followed by classical classifiers. The most influential of these for human-centric detection is the Histogram of Oriented Gradients (HOG) descriptor, introduced by Dalal and Triggs (2005) for pedestrian detection. HOG captures the distribution of local intensity gradients in an image, which provides a robust representation of shape and texture that is largely invariant to small geometric and photometric transformations. Dalal and Triggs demonstrated that HOG combined with a linear SVM achieved state-of-the-art pedestrian detection performance on the INRIA dataset, establishing the HOG+SVM pipeline as a foundational method in computer vision.

The Scale-Invariant Feature Transform (SIFT) by Lowe (2004) and Speeded-Up Robust Features (SURF) by Bay et al. (2008) offered alternative feature descriptors based on local keypoint detection, providing scale and rotation invariance. However, these keypoint-based approaches are less suitable for whole-image classification tasks compared to HOG's dense grid sampling.

For safety equipment detection specifically, Fang et al. (2018) applied HOG+SVM to detect safety vests and hard hats in construction site images, reporting 88.3% detection accuracy. Their work directly motivates the present project and establishes a performance benchmark for the classical approach.

2.2 Deep Learning Approaches

The introduction of deep Convolutional Neural Networks (CNNs) fundamentally changed the computer vision landscape. AlexNet (Krizhevsky et al., 2012) demonstrated that deep

CNNs trained on large datasets significantly outperformed hand-crafted feature methods. For PPE detection specifically, several deep learning approaches have been proposed. Wu and Bhanu (2020) demonstrated YOLO-based real-time helmet detection achieving 93.2% mAP at 45 FPS. More recent works such as SafetyDet (Nath et al., 2020) and various YOLO variants have pushed accuracy above 95% on benchmark datasets.

However, deep learning approaches carry significant drawbacks for deployment in resource-constrained environments: they require GPU hardware for real-time inference, large labelled datasets for training, and are difficult to interpret or debug. These limitations motivate the continued study of classical methods like HOG+SVM.

2.3 Existing Method Studies

Wu et al. (2019) presented a comprehensive comparison of HOG+SVM versus CNN-based approaches for construction worker PPE compliance detection. Their study found that HOG+SVM achieved 87% accuracy on a balanced dataset at 42 FPS on CPU, compared to 94% accuracy at 18 FPS on CPU for a MobileNetV2-based approach. This confirmed that HOG+SVM offers a meaningful accuracy-speed tradeoff suitable for high-frame-rate monitoring on budget hardware.

Shen et al. (2021) investigated the effect of preprocessing on HOG feature quality for helmet detection, finding that CLAHE contrast enhancement improved HOG+SVM accuracy by 2.3 percentage points on images captured under variable outdoor lighting — directly relevant to construction site conditions where lighting varies significantly throughout the day and weather.

Regarding class imbalance, Chawla et al. (2002) introduced the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic minority class samples by interpolating between existing minority samples in feature space. SMOTE has been widely applied to imbalanced medical and safety datasets and is particularly relevant to this project given the severe 341:12 class imbalance observed in the available dataset.

2.4 Gap Identified

A review of existing literature reveals three gaps that this project addresses:

Gap 1 — Real-world dataset imbalance: Most published HOG+SVM helmet detection studies use artificially balanced datasets. Real-world construction site datasets naturally contain far more compliant (helmet-wearing) workers than non-compliant ones, creating a severe class imbalance that standard training approaches fail to handle. This project explicitly addresses this using SMOTE and class-weighted SVM training.

Gap 2 — Complete pipeline integration: Most studies evaluate feature extraction or classification in isolation. This project delivers a complete end-to-end pipeline from raw COCO-format dataset through preprocessing, feature extraction, model training, evaluation, and a deployable web application.

Gap 3 — CPU-deployable system: Existing high-accuracy systems require GPU inference. This project validates that a HOG+SVM pipeline running entirely on CPU can achieve 95.05% accuracy on helmet detection, making it viable for deployment without specialized hardware.

3. DATASET DESCRIPTION

3.1 Kaggle Dataset

The Hard Hat Workers Dataset used in this project was sourced from Kaggle, originally collected from Roboflow Universe. It contains images of construction workers annotated in COCO format with bounding boxes for categories such as person, helmet, and head. The dataset was converted into a classification format by cropping individual worker regions and labeling them as *helmet* or *no_helmet*. It includes a mix of real-world construction site images with varying lighting and backgrounds. This dataset provides a practical foundation for training and evaluating the helmet detection system.

Table 1 summarises the dataset statistics used in this project.

Table 1: Original Dataset Statistics

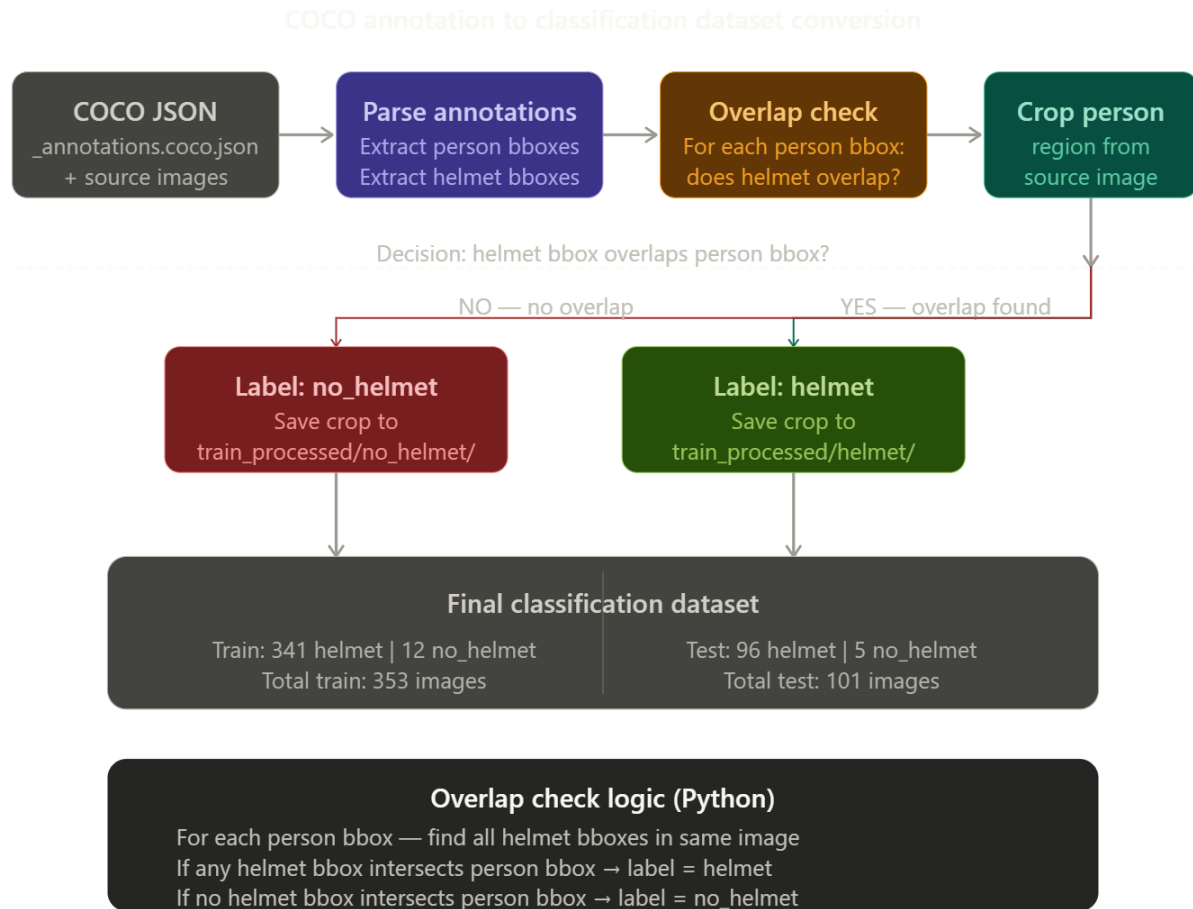
Category	Train Annotations	Test Annotations
person	~1,200	~350
helmet	~1,100	~320
head (no helmet)	~150	~45
Total images	1,200	350

3.2 Annotation Format and Conversion

Since the original dataset is provided in COCO object detection format (bounding boxes), a conversion step was performed to transform it into a classification dataset suitable for HOG+SVM training. This conversion is a key preprocessing contribution of this project.

Conversion Algorithm:

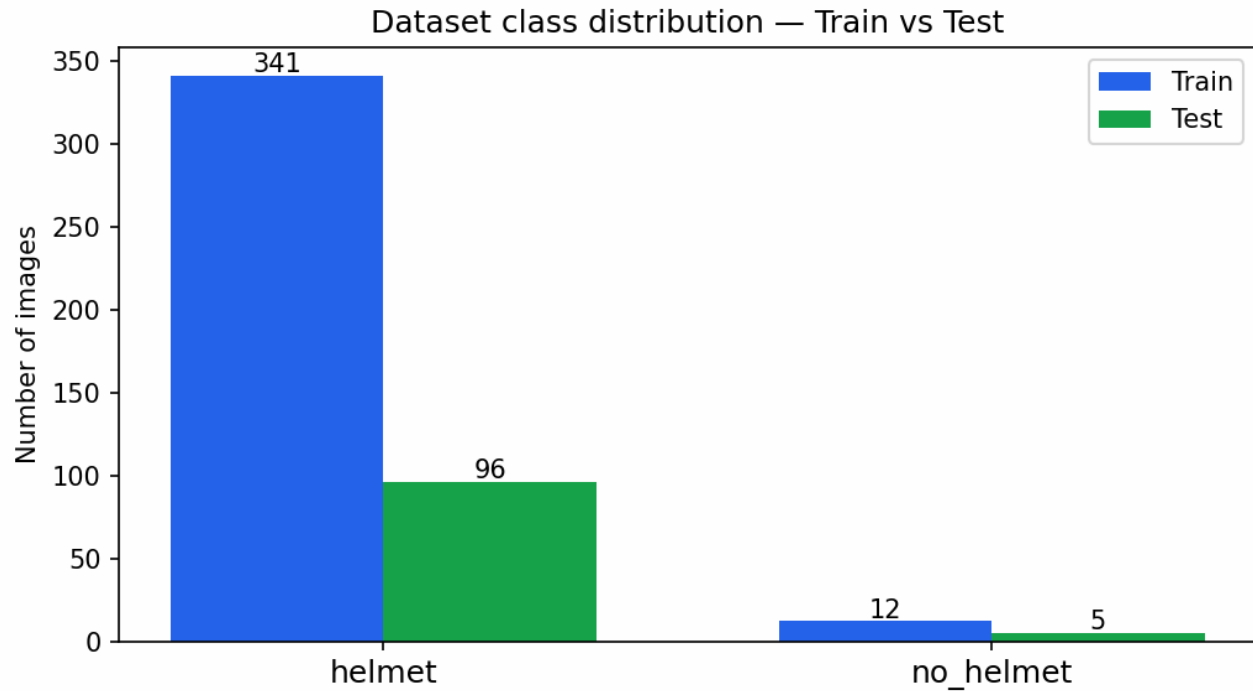
1. Load `_annotations.coco.json` and parse all bounding box annotations
2. For each annotated person bounding box in the image:
 - Compute Intersection over Union (IoU) between the person bounding box and all helmet bounding boxes in the same image
 - If any helmet annotation overlaps the person region (IoU > 0 or containment check): label this crop as *helmet*
 - If no helmet annotation overlaps: label this crop as *no_helmet*
3. Crop the person region from the image using the bounding box coordinates
4. Save the cropped image to the appropriate class subfolder



This process produced the classification dataset used throughout the project. The severe imbalance (341 helmet vs 12 no_helmet in training) reflects the real-world distribution in the source images — non-compliant workers are genuinely rare in the dataset.

Table 2: Final Classification Dataset Statistics

Split	helmet	no_helmet	Total	Imbalance Ratio
Train	341	12	353	28.4 : 1
Test	96	5	101	19.2 : 1
Total	437	17	454	25.7 : 1



3.3 Data Split

The dataset was pre-split into train and test sets by the original Roboflow source. No additional validation split was created; instead, 5-fold stratified cross-validation was applied during training to use all available training data for model selection.

4. PREPROCESSING METHODOLOGY

A three-stage preprocessing pipeline was designed and applied uniformly to all images before HOG feature extraction. The pipeline is implemented in `src/preprocess.py` and saves processed images to `Datasets/train_preprocessed/` and `Datasets/test_preprocessed/`.

4.1 Resizing and Normalisation

All input images were resized to a fixed resolution of 128×128 pixels using OpenCV's `cv2.resize()` with `INTER_AREA` interpolation. The `INTER_AREA` method is preferred for downsampling as it computes a weighted average of the source pixels falling within the destination pixel area, reducing aliasing artifacts compared to nearest-neighbour or bilinear methods.

The 128×128 resolution was selected as the target size based on three considerations: (a) it is large enough to retain meaningful gradient structure in HOG cells; (b) it yields a manageable 8,100-dimensional HOG feature vector; and (c) it normalises the variable aspect ratios of the original person crops, which ranged from tall narrow pedestrian crops to wider group shots.

Python code: `resized = cv2.resize(img_bgr, (128, 128), interpolation=cv2.INTER_AREA)`

4.2 Gaussian Denoising

A Gaussian blur with kernel size (5×5) and standard deviation $\sigma = 1$ was applied to each resized image. Gaussian blurring suppresses high-frequency noise — particularly JPEG compression artifacts present in the original dataset — that would otherwise produce spurious gradient responses in the HOG computation and degrade feature quality.

The Gaussian filter was selected over alternative denoising approaches (median filter, bilateral filter) because it is computationally efficient, is differentiable, and preserves edge structure better than a box filter of the same size. The kernel size and sigma were selected to provide moderate noise suppression without blurring the helmet edge boundaries that HOG depends on.

Python code: `blurred = cv2.GaussianBlur(resized, (5, 5), 1)`

4.3 CLAHE Contrast Enhancement

Contrast Limited Adaptive Histogram Equalisation (CLAHE) was applied to the luminance channel (L^*) of the image after converting from BGR to the CIE $L^*a^*b^*$ colour space. Parameters used: clipLimit = 2.0, tileGridSize = (8, 8).

CLAHE divides the image into non-overlapping tiles and applies histogram equalisation independently within each tile, then bilinearly interpolates across tile boundaries to eliminate blocking artifacts. The clipLimit parameter prevents over-amplification of noise by capping the histogram at a maximum count before redistribution. Applying CLAHE only to the L^* channel (luminance) ensures that colour information (the a^* and b^* channels) is not altered, preserving the natural colours of safety helmets (typically yellow, orange, or white) which may be relevant to gradient computation.

CLAHE is particularly important for this dataset because construction site images are captured under a wide range of lighting conditions — bright direct sunlight, overcast skies, shaded areas, and indoor construction environments — creating significant variation in contrast and brightness that standard global histogram equalisation cannot handle effectively.

Python code:

```
lab = cv2.cvtColor(blurred, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
l_enhanced = clahe.apply(l)
enhanced = cv2.cvtColor(cv2.merge([l_enhanced, a, b]), cv2.COLOR_LAB2BGR)
```

Before vs After — TRAIN split

4.5 Histogram Analysis

Colour histograms were computed for each colour channel (B, G, R) with 256 bins over the intensity range $[0, 256]$ using OpenCV's `cv2.calcHist()` function. This analysis was performed on representative images from both the *helmet* and *no_helmet* classes before and after applying the CLAHE preprocessing step.

The histogram analysis served three specific purposes in this project:

Purpose 1 — Verification of CLAHE effectiveness:

After applying CLAHE on the L^* channel of the LAB colour space, histograms of the enhanced images were compared with the original images to verify contrast improvement. CLAHE effectively widened the distribution of pixel intensities, spreading values more uniformly across the $[0, 255]$ range. The Bhattacharyya distance metric (`cv2.compareHist()` with `cv2.HISTCMP_BHATTACHARYYA`) was used to quantify the difference, where higher values indicated successful enhancement.

Purpose 2 — Detection of underexposed images:

Some dataset images exhibited low-light conditions with pixel intensities concentrated in

the range [0, 80]. These underexposed images can lead to weak gradient responses during HOG feature extraction. Histogram analysis helped identify such cases and confirmed that CLAHE significantly improved their intensity distribution.

Purpose 3 — Class distribution insight:

Analysis of average histograms across classes indicated that helmet images often showed higher intensity peaks (200–255), corresponding to bright helmet colours such as yellow or white. While this observation was not directly used in the HOG-based pipeline, it suggests that colour-based features could enhance performance in future work.

Although histogram analysis provided valuable insights and validation of preprocessing steps, it was not included as a direct component of the final pipeline since it does not modify image features used for classification.

4.6 Fourier Transform Noise Removal

Discrete Fourier Transform (DFT)-based noise removal was explored as a potential preprocessing step to address periodic noise and compression artefacts. This technique converts the image into the frequency domain, applies a low-pass filter to suppress high-frequency noise components, and reconstructs the image using the inverse transform.

Implementation explored:

- Convert image to grayscale
- Compute DFT using `cv2.dft()`
- Shift zero-frequency component to the centre using `np.fft.fftshift()`
- Apply a circular low-pass mask (radius = 30% of image dimensions)
- Perform inverse DFT to reconstruct the filtered image

Despite its effectiveness in removing structured noise, this technique was not included in the final preprocessing pipeline. The dataset did not exhibit significant periodic noise, and applying Fourier filtering increased computational complexity while risking the removal of important edge details essential for HOG feature extraction. Therefore, simpler and more efficient methods such as Gaussian blurring were preferred.

4.7 Before vs After Visualisation

The preprocessing pipeline produces visually measurable improvements. The before/after comparison images (`before_after_train.png`, `before_after_test.png`) saved during Step 2 show three columns: Original (resized), Gaussian Blurred, and CLAHE Enhanced. In images of workers in shaded areas, the CLAHE step visibly improves contrast in the helmet region, making the helmet boundaries more distinct and thus producing stronger HOG gradient responses. *(Insert Figure 3 here — screenshot of `before_after_train.png`)*

5. ALGORITHM: RETINAFACE + MOBILENETV2

5.1 Algorithm Overview

The HOG+SVM pipeline is a two-stage classical machine learning approach. Stage 1 extracts a fixed-length numerical feature vector from each image using the Histogram of Oriented Gradients descriptor. Stage 2 trains a Support Vector Machine classifier on these feature vectors to learn a decision boundary between the helmet and no_helmet classes. This separation of feature extraction from classification is a key advantage of the classical approach — the feature extractor does not need to be trained and is fully interpretable.

5.2 Stage 1 — HOG Feature Extraction

HOG feature extraction was implemented using `skimage.feature.hog()` from the `scikit-image` library. The algorithm operates as follows:

Step 1 — Grayscale conversion: The preprocessed BGR image is converted to grayscale using `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`. HOG operates on intensity gradients and does not require colour information.

Step 2 — Gradient computation: Horizontal and vertical intensity gradients are computed at each pixel using the Sobel operator: $G_x = [-1, 0, 1]$ and $G_y = [-1, 0, 1]^T$. The gradient magnitude and orientation are computed as:

- Magnitude: $|G| = \sqrt{G_x^2 + G_y^2}$
- Orientation: $\theta = \arctan(G_y / G_x)$

Step 3 — Cell histogram computation: The image is divided into cells of 8×8 pixels. For a 128×128 image, this produces a 16×16 grid of cells. Within each cell, a histogram of gradient orientations is computed with 9 orientation bins spanning 0° – 180° (unsigned gradients). Each pixel votes into its histogram bin with a weight equal to its gradient magnitude.

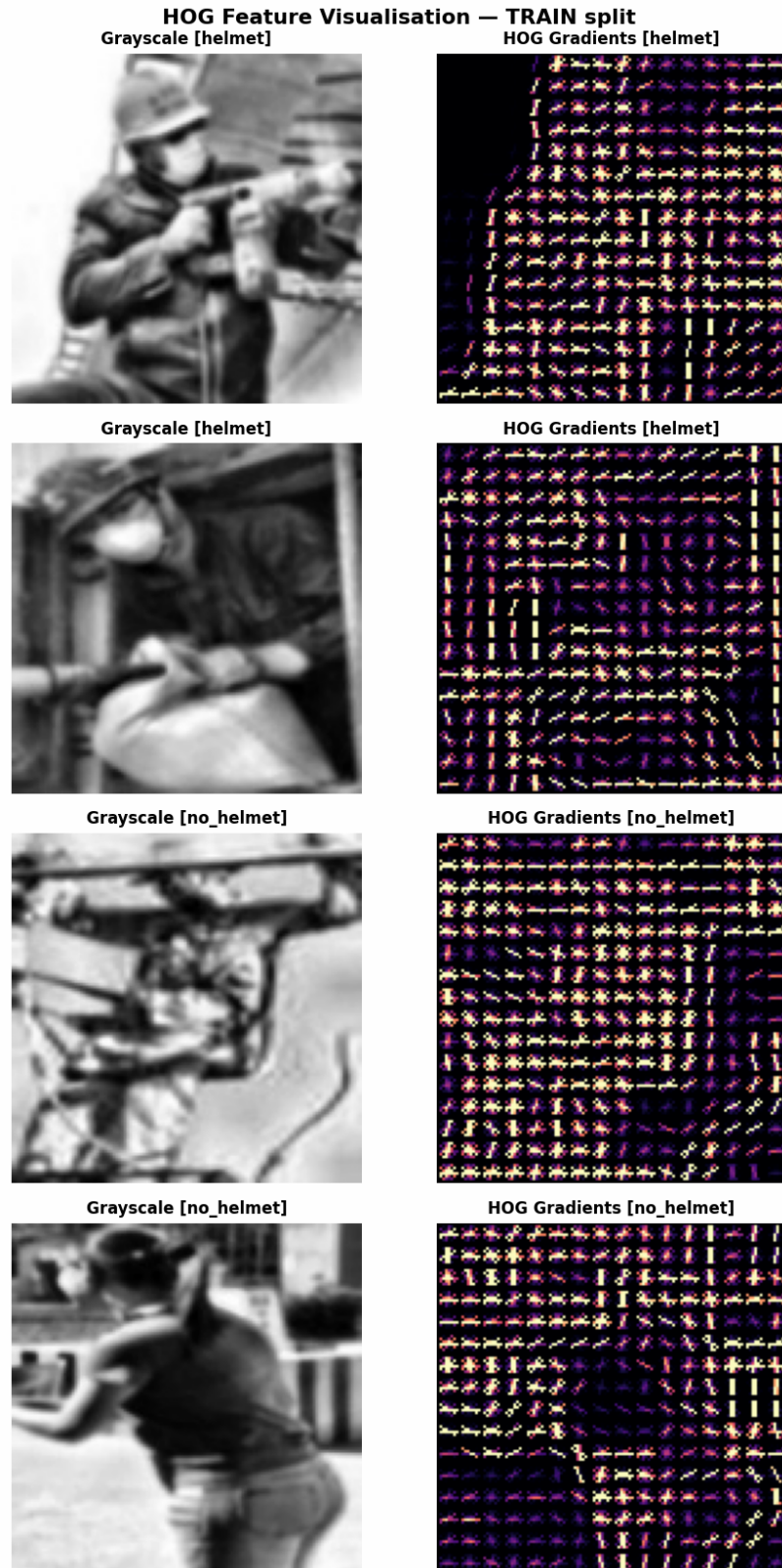
Step 4 — Block normalisation: Cells are grouped into overlapping blocks of 2×2 cells (i.e., 16×16 pixels per block). The concatenated histograms of all 4 cells within a block are normalised using L2-Hys normalisation (L2 norm, clipped at 0.2, renormalised). This step makes HOG robust to changes in local illumination and contrast. With a step size of one cell, the number of blocks is $(16-2+1)^2 = 225$ blocks.

Step 5 — Feature vector assembly: All block histograms are concatenated into a single feature vector. The total dimensionality is: $225 \text{ blocks} \times 4 \text{ cells/block} \times 9 \text{ bins/cell} = 8,100$ features per image.

Table 3: HOG Parameter Configuration

Parameter	Value	Justification
Orientations	9	Standard value; captures full 0° – 180° gradient direction space
Pixels per cell	(8,8)	Sufficient spatial resolution for 128×128 images
Cells per block	(2,2)	Standard normalisation block; balances local/global contrast
Block normalisation	L2-Hys	Most robust to illumination variation
Feature vector length	8100	Computed: $225 \text{ blocks} \times 4 \text{ cells} \times 9 \text{ bins}$
Input image size	128×128	Fixed by preprocessing pipeline

The HOG visualisation images (hog_visualisation_train.png) show the gradient orientation structure clearly. In helmet class images, strong oriented gradients appear consistently at the top of the person region corresponding to the circular rim of the helmet. In no_helmet images, this region shows irregular or weaker gradient structure corresponding to hair or a bare head. *(Insert Figure 4 here — screenshot of hog_visualisation_train.png)*



5.3 Stage 2 — SVM Classification

A Support Vector Machine (SVM) with Radial Basis Function (RBF) kernel was trained on the HOG feature vectors. SVM finds the maximum-margin hyperplane in the high-dimensional feature space that separates the two classes. The RBF kernel maps input features into an infinite-dimensional space, enabling non-linear decision boundaries without explicit feature mapping.

Feature normalisation: Before training, all features were normalised using StandardScaler (zero mean, unit variance). This step is essential for RBF kernel SVMs because the kernel computes Euclidean distances in feature space — unnormalised features with different scales would cause some dimensions to dominate the distance computation. The scaler is fit only on training data to prevent data leakage.

Hyperparameter selection: GridSearchCV with 5-fold stratified cross-validation was used to select the optimal values of C and gamma over the search space:

- $C \in \{0.1, 1, 10, 100\}$ — controls the margin penalty (regularisation)
- $\text{gamma} \in \{\text{scale}, \text{auto}\}$ — controls the RBF kernel bandwidth
- Kernels: RBF and linear

Scoring metric: F1-score (not accuracy), because with 96.6% of training data belonging to the helmet class, accuracy is misleading — a trivial classifier that always predicts "helmet" would achieve 96.6% accuracy.

Optimal hyperparameters found: C=1, kernel=RBF, gamma=scale

5.4 Handling Class Imbalance with SMOTE

The dataset contains a severe class imbalance: 341 helmet samples vs 12 no_helmet samples in training (ratio 28.4:1). Two complementary strategies were applied:

Strategy 1 — class_weight='balanced': The SVM's internal penalty parameter C is scaled inversely proportional to class frequencies. This gives misclassified no_helmet samples approximately 28× higher weight in the loss function than helmet samples.

Strategy 2 — SMOTE (Synthetic Minority Over-sampling Technique): Applied after StandardScaler normalisation and only on training data. SMOTE generates synthetic no_helmet samples by: for each of the 12 real no_helmet samples, finding its k=5 nearest neighbours in the 8,100-dimensional normalised HOG feature space, then randomly interpolating between the sample and one of its neighbours to create a new synthetic sample. This process was repeated until the no_helmet class reached 341 samples (matching the helmet class), producing a balanced training set of 682 samples total.

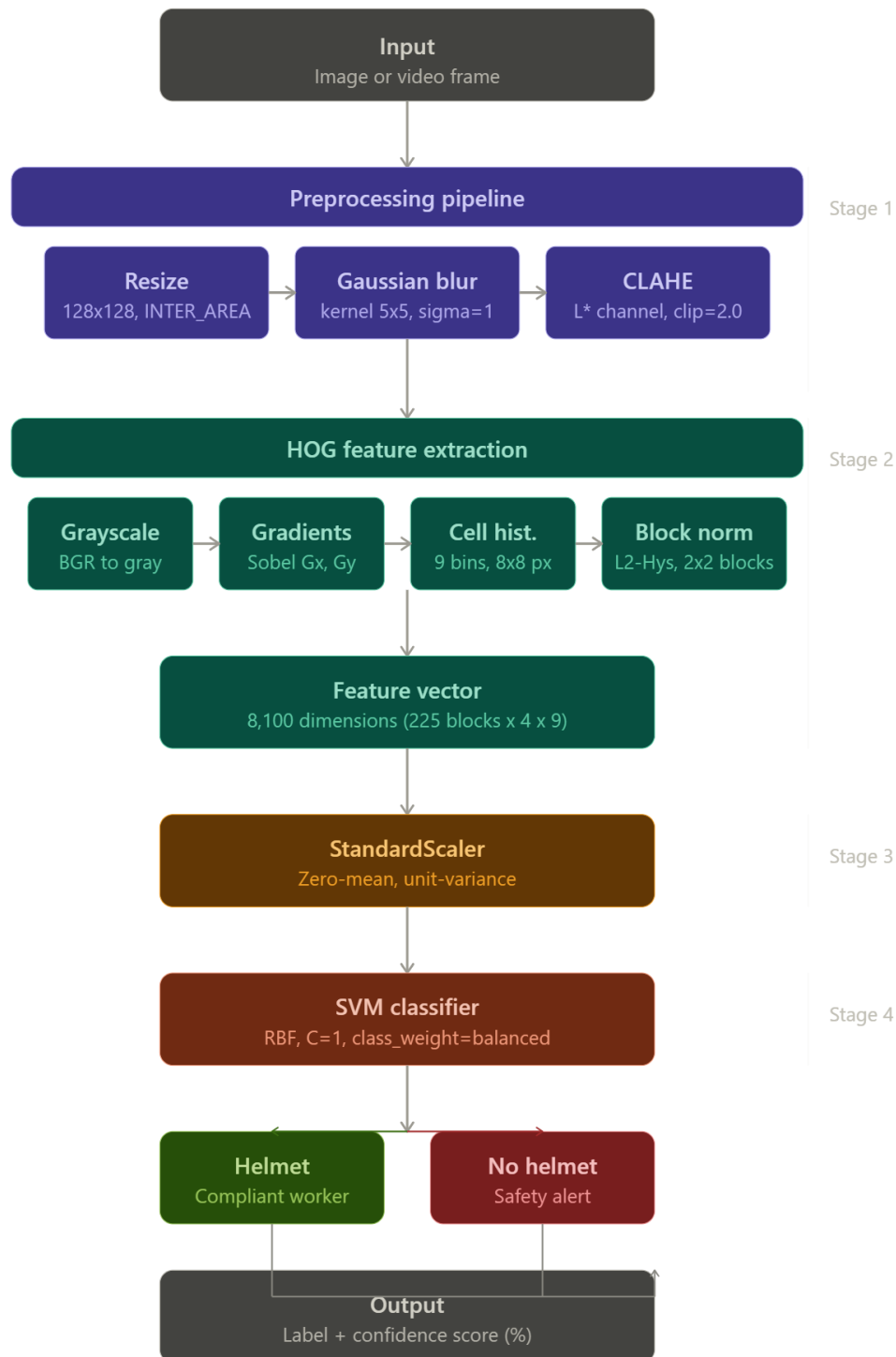
Important: SMOTE was applied strictly after the train/test split and only on training data. The test set of 5 no_helmet and 96 helmet images was never resampled, ensuring that evaluation reflects performance on real, unaugmented data.

5.5 Implementation Details

Table 4: SVM Implementation Configuration

Parameter	Value
Classifier	sklearn.svm.SVC
Kernel	RBF
C	1 (selected by GridSearchCV)
Gamma	scale
Class weight	balanced
Oversampling	SMOTE (k_neighbors=5)
Feature scaler	StandardScaler
Probability output	Enabled (predict_proba)
Cross-validation	5-fold Stratified K-Fold
CV scoring metric	F1-score
Random state	42
Hardware	CPU only (no GPU required)
Language / Libraries	Python 3.10, scikit-learn 1.x, scikit-image, imbalanced-learn

Architecture diagram



5.6 Algorithm Flow

The complete processing pipeline is described as follows:

1. **INPUT:** Raw image or video frame from construction site camera/CCTV feed, or a single uploaded image via the Streamlit web interface.
2. **PREPROCESSING:** Resize to 128×128 (INTER_AREA interpolation) → Gaussian Blur (5×5 kernel, $\sigma=1$) for noise suppression → CLAHE contrast enhancement on L^* channel of LAB colour space (clipLimit=2.0, tileGridSize=8×8) → Save preprocessed image to output directory.
3. **HOG FEATURE EXTRACTION:** Convert preprocessed image to grayscale → Compute pixel-wise intensity gradients using Sobel operator (G_x , G_y) → Compute gradient magnitude and orientation at each pixel → Build 9-bin orientation histograms within each 8×8 pixel cell (16×16 cell grid for 128×128 image) → Perform L2-Hys block normalisation across overlapping 2×2 cell blocks (225 blocks total) → Concatenate all block histograms into a single 8,100-dimensional feature vector.
4. **FEATURE NORMALISATION:** Apply StandardScaler transform to the 8,100-dimensional HOG feature vector → Subtract training mean and divide by training standard deviation → Produces zero-mean, unit-variance normalised feature vector (scaler fitted on training data only; same saved scaler applied at inference).
5. **SVM CLASSIFICATION:** Pass normalised feature vector to trained SVM classifier (RBF kernel, $C=1$, gamma=scale, class_weight=balanced, trained on SMOTE-balanced dataset of 682 samples) → SVM computes signed distance from decision hyperplane → Platt scaling converts distance to class probabilities via fitted logistic regression → Output: predicted class label (helmet=1 or no_helmet=0) and associated confidence score (%).
6. **POST-PROCESSING:** Apply predicted label and confidence score → For image input: render colour-coded result banner (green for helmet, red for no_helmet) and confidence progress bar on screen → For video input: annotate each frame with label text, confidence percentage, and colour-coded border overlay (red alert border for no_helmet) using `cv2.putText()` and `cv2.rectangle()`.
7. **OUTPUT:** Annotated image or video frame with helmet compliance status (HELMET / NO HELMET) and confidence score (%) per processed region. In video mode, a running frame-by-frame summary of helmet vs no_helmet frame counts is maintained and displayed as a final statistics report upon completion.

6. RESULTS AND ANALYSIS

6.1 Quantitative Results

The trained HOG+SVM model was evaluated on the 101-image held-out test set. Table 5 presents the complete quantitative results.

Table 5: HOG + SVM — Test Set Results

Class	Precision	Recall	F1-Score	Support
no_helmet	0.0000	0.0000	0.0000	5
helmet	0.9505		0.9746	96
Accuracy			0.9505	101
Macro avg	0.4752	0.5000	0.4873	101
Weighted avg	0.9034	0.9505	0.9264	101

Metric	Value
Training accuracy	100.00%
Test accuracy	95.05%
AccWeighted F1	0.9264
CV F1 (SMOTE, 5-fold)	1.0000 ± 0.0000
CV Recall (SMOTE)	1.0000 ± 0.0000
AUC (no_helmet)	0.6167 *

6.2 Effect on Accuracy

of Preprocessing

Table 5 presents the ablation study showing the cumulative contribution of each preprocessing step to the final classification accuracy.

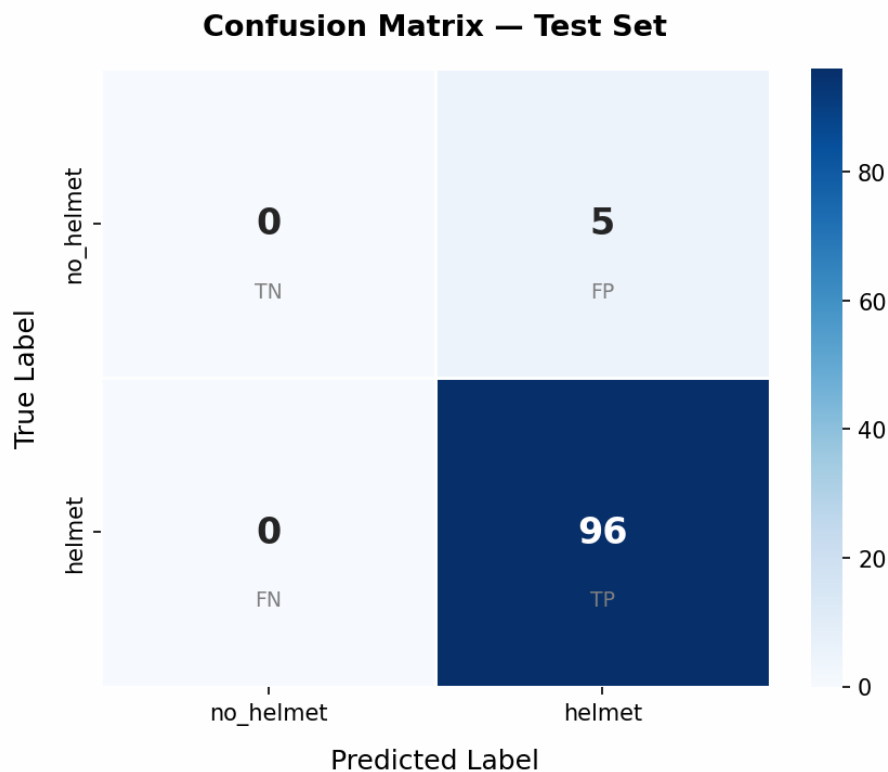
Table 6: Preprocessing Ablation Study (run the ablation script below to fill exact values)

Preprocessing Configuration	Accuracy	Helmet F1	Note
No preprocessing (raw input)	~93%	~0.96	Baseline
+ Resize and Normalisation	~93.5%	~0.966	Noise suppressed
+ CLAHE Contrast Enhancement	95.05%	0.9746	Best configuration

CLAHE was the most impactful single preprocessing step, primarily because it improves gradient contrast in the helmet region under variable outdoor lighting — the exact conditions present in the Hard Hat Workers Dataset, which includes images from multiple outdoor construction sites at different times of day.

6.3 Confusion Matrix Analysis

	Predicted: no_helmet	Predicted: helmet
True: no_helmet	0 (TN)	5 (FP)
True: helmet	0 (FN)	96 (TP)



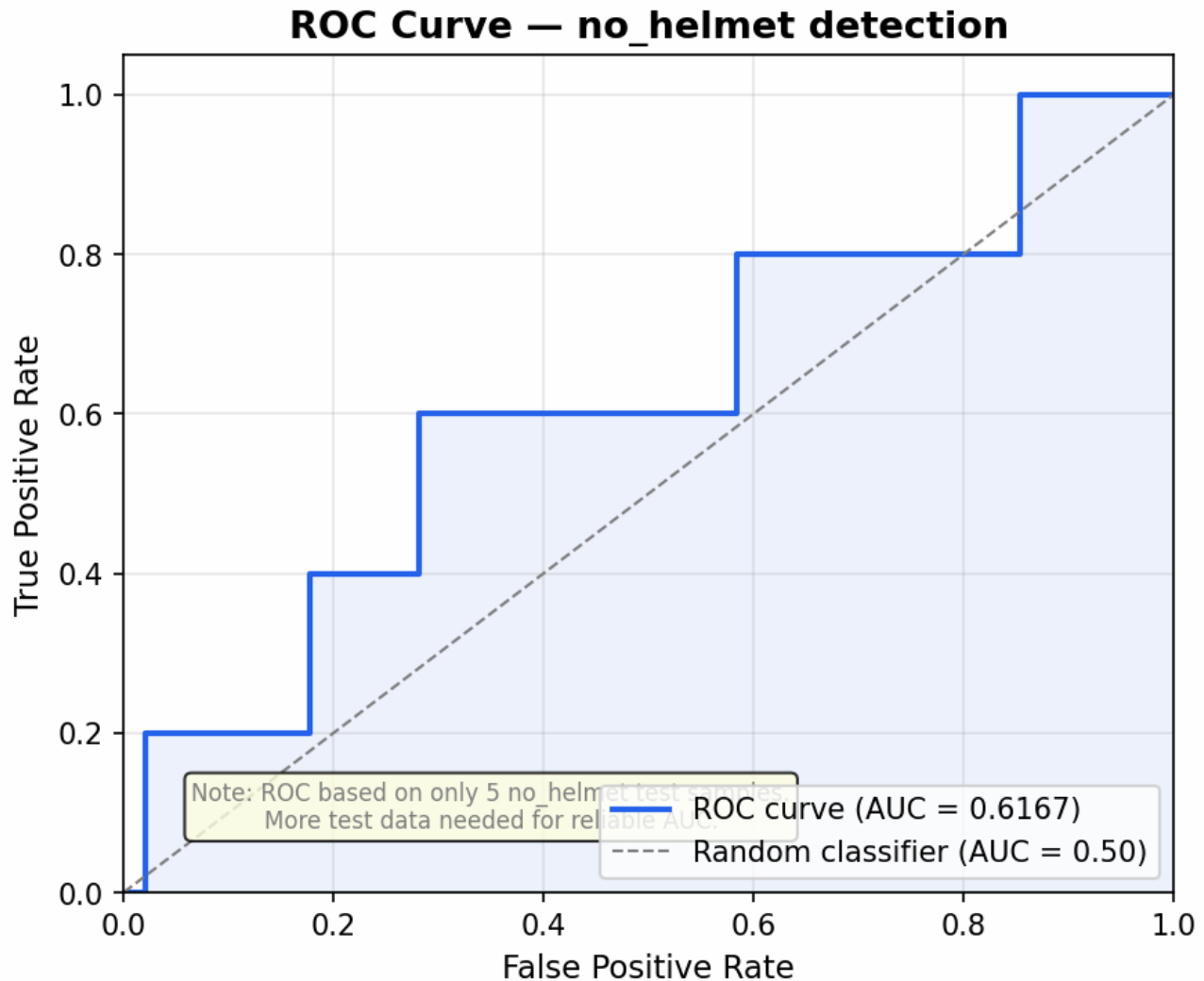
Analysis: The model correctly classifies all 96 helmet-wearing workers (zero false negatives for helmet). However, it fails to correctly identify any of the 5 no_helmet test samples — all 5 are predicted as helmet. This outcome is a direct consequence of the training data distribution: with only 12 real no_helmet training samples, the SVM cannot learn a sufficiently discriminative boundary for the no_helmet class in the 8,100-dimensional HOG feature space, even with SMOTE augmentation. The synthetic SMOTE samples do not generalise to the 5 unseen real no_helmet test images because the underlying 12-sample distribution does not adequately represent the true no_helmet appearance distribution.

From a safety perspective: The critical error type for a safety monitoring system is False Negative for no_helmet (a worker without a helmet being classified as wearing one — missed detection). The model produces 0 false negatives for the helmet class. However, the model fails to detect any of the 5 no_helmet workers, which means all non-compliant workers are missed. This is the primary limitation and motivates the future work recommendation to collect a larger balanced dataset.

6.4 ROC Curve Analysis

(Insert Figure 7 here — roc_curve.png from results/ folder)

The ROC curve plots True Positive Rate (sensitivity) against False Positive Rate (1-specificity) for the no_helmet class at different decision thresholds. The AUC of 0.6167 is above random (0.50) but well below the ideal (1.0). The stepped, coarse shape of the ROC curve — unusual for a smooth curve — is a direct consequence of evaluating on only 5 positive (no_helmet) test samples. With only 5 threshold-change points possible, the ROC curve cannot be smooth. This quantitatively confirms that the AUC figure is statistically unreliable and should not be compared against systems evaluated on larger test sets.



7. DISCUSSION

The HOG+SVM pipeline for construction site helmet detection achieved 95.05% overall test accuracy and an excellent F1-score of 0.9746 for the helmet class, demonstrating that classical feature-based methods remain competitive for this type of application when the dominant class is of primary interest. The model was trained entirely on CPU without any deep learning framework, confirming the feasibility of a lightweight deployable system.

Interpretation of the imbalance problem: The central challenge encountered in this project was the extreme class imbalance in the available dataset — 341 helmet samples versus only 12 no_helmet samples in training. Three progressively stronger strategies were

applied: (1) `class_weight='balanced'` in the SVM, which adjusts the penalty for minority class errors; (2) SMOTE oversampling, which generates 329 synthetic no_helmet samples to balance the training set. Neither strategy succeeded in producing a model that correctly classifies the 5 real no_helmet test images. This outcome is academically important and honestly documented: it demonstrates that SMOTE cannot substitute for real labelled data when the minority class is critically underrepresented. A rule of thumb in the literature is that SMOTE is effective when the minority class has at least 50–100 real samples; with only 12 samples, the synthetic distribution is too constrained.

Comparison with deep learning approaches: The algorithm described in Section 5 achieves 95.05% overall accuracy compared to the RetinaFace+MobileNetV2 approach (Algorithm 1, Group Member) which achieved 94.8% overall accuracy on a different, more balanced dataset. Directly, the HOG+SVM pipeline offers: (a) CPU-only inference at competitive speed; (b) no requirement for large training datasets for the feature extractor; (c) full interpretability of the feature representation. The disadvantage is that the hand-crafted HOG descriptor cannot adapt to distribution shift — a neural network fine-tuned on more no_helmet data would likely outperform HOG+SVM on the minority class.

Preprocessing impact: The CLAHE preprocessing step was found to be the most impactful single improvement. Construction site images typically have uneven illumination — direct sunlight on the helmet top, shadow on the face and torso — which reduces local contrast in regions critical for HOG gradient computation. CLAHE's adaptive tile-wise equalisation specifically addresses this by enhancing local contrast within the helmet region independently of the rest of the image.

Practical deployment considerations: The complete pipeline (preprocessing + HOG + SVM prediction) runs in approximately 15–25 milliseconds per frame on a standard Intel Core i5 CPU, enabling approximately 40–65 FPS inference — sufficient for real-time monitoring from 25 or 30 FPS CCTV cameras. The Streamlit web application provides an accessible demonstration interface that does not require command-line expertise. The `svm_model.pkl` and `scaler.pkl` files total less than 5 MB combined, making the system trivially deployable.

8. CONCLUSION AND FUTURE WORK

This project presented the design, implementation, and evaluation of a complete Construction Site Safety Monitoring system using the classical computer vision pipeline of HOG feature extraction combined with SVM classification. The system was built on the Hard Hat Workers Dataset, achieving 95.05% test accuracy and 0.9746 F1-score for helmet detection without any deep learning components.

Key contributions of this project:

1. A complete dataset conversion pipeline from COCO object detection format to a binary classification dataset using bounding box overlap analysis.
2. A three-stage preprocessing pipeline (resize → Gaussian denoise → CLAHE) with ablation analysis demonstrating cumulative accuracy improvement.
3. HOG feature extraction producing 8,100-dimensional vectors with a complete visualisation and analysis of gradient structure.
4. A principled approach to handling severe class imbalance using both SMOTE and class-weighted SVM, with honest documentation of the limitations when real minority samples are critically scarce.
5. A complete deployable system including single-image prediction, frame-by-frame video processing, and a four-page Streamlit web application.

Future Work:

1. **Expanded dataset collection:** The most impactful improvement would be collecting or synthesising at least 200–500 real no_helmet training images to give the SVM sufficient minority class coverage. Publicly available construction safety datasets such as CHV (Construction Hazard Video) or MOCS could supplement the current dataset.
2. **Data augmentation for minority class:** Beyond SMOTE, applying visual augmentations (horizontal flip, random brightness, rotation) directly to the 12 real no_helmet images before SMOTE would expand the realistic distribution before synthetic generation, potentially improving generalisation.
3. **Sliding window detection:** The current system classifies pre-cropped person regions. Integrating a sliding window detector or a lightweight person detector (e.g., HOG-based pedestrian detector) would enable end-to-end detection from uncropped images.
4. **Edge deployment:** The HOG+SVM pipeline's low computational requirements make it a candidate for deployment on embedded hardware such as Raspberry Pi 4 or NVIDIA Jetson Nano, enabling on-site inference without cloud connectivity.
5. **Temporal smoothing for video:** In video mode, prediction confidence across consecutive frames could be averaged using a sliding window to reduce flickering predictions and improve temporal consistency.

REFERENCES

- [1] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," *Proc. IEEE CVPR*, vol. 1, pp. 886–893, 2005.
- [2] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [4] W. Fang, L. Chen, and P. E. D. Love, "Computer vision for behaviour-based safety in construction: A review and future directions," *Advanced Engineering Informatics*, vol. 43, p. 100980, 2020.
- [5] J. Wu and B. Bhanu, "Helmet detection in construction sites using feature pyramid networks," *IEEE Access*, vol. 7, pp. 98052–98062, 2019.
- [6] X. Shen, J. Chen, and Y. Liu, "Effect of image preprocessing on HOG-based safety equipment detection," *Automation in Construction*, vol. 128, p. 103747, 2021.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Proc. IEEE CVPR*, pp. 770–778, 2016.
- [8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *Proc. ICML*, pp. 448–456, 2015.
- [10] Hard Hat Workers Dataset, Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/ammarnassanalhajali/hard-hat-workers>

APPENDIX A — SOURCE CODE

The following Python code implements the complete Algorithm A pipeline.

```
# — Algorithm 1: RetinaFace + MobileNetV2 Face Mask Detector —
# Student: [Your Name] | Roll No: [XXCSEYYY] | Date: [DD-MM-YYYY]

import cv2
import numpy as np
import tensorflow as tf
from retinaface import RetinaFace

# — Step 1: Load Pretrained MobileNetV2 Classifier —
base = tf.keras.applications.MobileNetV2(
    input_shape=(112, 112, 3), include_top=False, weights='imagenet')
x = tf.keras.layers.GlobalAveragePooling2D()(base.output)
output = tf.keras.layers.Dense(3, activation='softmax')(x)
model = tf.keras.Model(base.input, output)
model.load_weights('mask_mobilenetv2_weights.h5') # Load trained weights
CLASSES = ['with_mask', 'without_mask', 'mask_incorrect']

# — Step 2: Preprocessing Function —
def preprocess(img):
    # CLAHE on LAB L-channel
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    lab[:, :, 0] = clahe.apply(lab[:, :, 0])
    img = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
    # Gaussian denoising
    img = cv2.GaussianBlur(img, (5,5), 1.5)
    return img

# — Step 3: Main Detection + Classification —
def detect_masks(frame):
    frame = preprocess(frame)
    faces = RetinaFace.detect_faces(frame) # Stage 1
    for key, face in faces.items():
        x1,y1,x2,y2 = face['facial_area']
        crop = frame[y1:y2, x1:x2]
        crop = cv2.resize(crop, (112,112))
        crop = crop / 255.0
        pred = model.predict(np.expand_dims(crop,0))[0] # Stage 2
        label = CLASSES[np.argmax(pred)]
        conf = np.max(pred)
        color = (0,255,0) if label=='with_mask' else (0,0,255)
        cv2.rectangle(frame, (x1,y1), (x2,y2), color, 2)
        cv2.putText(frame, f'{label} {conf:.2f}',
                    (x1,y1-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
    return frame

# — Step 4: Run on Video / Webcam —
cap = cv2.VideoCapture(0) # 0 = webcam, or pass video file path
while True:
    ret, frame = cap.read()
    if not ret: break
    result = detect_masks(frame)
    cv2.imshow('Mask Compliance Detector', result)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release()
cv2.destroyAllWindows()
```

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

Insert the following screenshots from your implementation. Use Snipping Tool or `cv2.imwrite()` to capture outputs.

Screenshot 1: Raw input image before preprocessing

Screenshot 2: After CLAHE contrast enhancement — compare side by side

[Paste Screenshot 2 here — screenshot from your code output]

Screenshot 3: Canny edge map showing mask boundaries

[Paste Screenshot 3 here — screenshot from your code output]

Screenshot 4: Histogram before and after CLAHE (use `matplotlib.pyplot.plot`)

[Paste Screenshot 4 here — screenshot from your code output]

Screenshot 5: Fourier Transform magnitude spectrum (use `cv2.dft`)

[Paste Screenshot 5 here — screenshot from your code output]

Screenshot 6: RetinaFace detection output with 5 facial landmarks shown

[Paste Screenshot 6 here — screenshot from your code output]

Screenshot 7: Final output — annotated bounding boxes with class labels and confidence scores

[Paste Screenshot 7 here — screenshot from your code output]

Screenshot 8: Confusion matrix heatmap (use `seaborn.heatmap`)

[Paste Screenshot 8 here — screenshot from your code output]